TOP 50 OOPS CONCEPTS QUESTIONS

Questions Every Developer Should Know

PAGE 1 : OOPS FUNDAMENTALS (Q1 - Q6)

(1) What is OOP and why is it used?

Answer: OOP (Object-Oriented Programming) is a programming paradigm based on the concept of "objects", which can contain data and code.

Why:

- Helps in writing clean, reusable and maintainable code.
- Follows real-world modeling.
- Supports scalability and flexibility.

Example (Real World):

Consider a "Car". It has properties (color, speed) and behaviors (start, stop). In OOP, we model this as a class and create objects.

Properties
- color
- speed



Behaviors
- start()
- stop()

(2) What are the four pillars of OOP?

Answer:

The four main pillars (principles) of OOP are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Quick View:

Encapsulation	Abstraction	Inheritance	Polymorphism
Binding data and methods together as a single unit.	Hiding internal details and showing only essential features.	Deriving new class from an existing class to reuse code.	One interface, multiple forms (many behaviors).

(3) What is the difference between a class and an object?

Answer:

- Class is a blueprint or template.
- Object is an instance of a class.
- Class is logical; Object is physical.

Example:

```
class Car {                                    // Class (Blueprint)
    String color;
    int speed;
    void start() { System.out.println("Car started"); }
}

Car c1 = new Car();          // Object (Instance)
Car c2 = new Car();          // Another Object
```

(4) What is encapsulation? Give a real-world example.

Answer:

Encapsulation is the concept of binding data (variables) and methods that operate on the data into a single unit (class) and restricting direct access to some of the object's components. It helps in data hiding and security.

Real-world Example:

A capsule → outer shell hides the medicine inside. Only through proper methods, we can access it.



Java Example:

```
class BankAccount {
  private double balance;          // data hiding
  public void setBalance(double b) { balance = b; }
  public double getBalance()  { return balance; }
}
```

(5) What is abstraction? How is it different from encapsulation?

Answer:

- Abstraction is the concept of hiding unnecessary details and showing only the essential features to the user.

Difference:

- Encapsulation = Hiding data (how things work)
- Abstraction = Hiding implementation (what it does)

Example:

```
abstract class Shape {
   abstract void area();     // only method signature
}
class Circle extends Shape {
   void area() {
        System.out.println("Calculating area of Circle");
   }
}
```

Key Idea: User knows what it does, not how it does it.

(6) What is inheritance and why do we use it?

Answer:

Inheritance is the mechanism of acquiring the properties (fields) and behaviors (methods) of one class (parent) into another class (child).

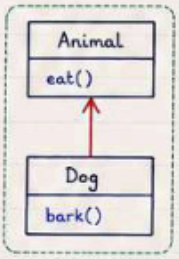
Why:

- Reusability of code.
- Establishes relationship between classes.
- Easy to extend and maintain the code.

Example:

```
class Animal {
   void eat() { System.out.println("Eating..."); }
}

class Dog extends Animal {
   void bark() { System.out.println("Barking..."); }
}
```



★ OOP is not just about code, it's about modeling real-world with clarity. ★

@aman_views

Top 50 OOPS Interview Questions

PAGE 2 - QUESTIONS 7 TO 12

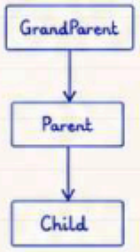
7. What are the different types of inheritance?

Answer:

- In Java, we have the following types of inheritance:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (through interfaces)
5. Hybrid Inheritance (combination of above)

Example (Multilevel)



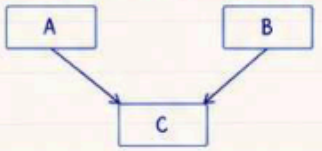
Pro Tip: Java doesn't support multiple inheritance through classes to avoid ambiguity.

8. Why doesn't Java support multiple inheritance through classes?

Answer:

- Multiple inheritance through classes can lead to ambiguity.
- If two parent classes have the same method and the child class inherits both, the JVM gets confused which method to call.
- To avoid this "Diamond Problem", Java supports multiple inheritance only through interfaces.

Diamond Problem (Example in other languages)



If A and B have same method show(), which one should C use?

9. What is polymorphism?

Answer:

- Polymorphism means "many forms".
- It allows an object to take many forms.
- In Java, it is achieved through method overloading (compile-time) and method overriding (runtime).

Example:

```
class Animal {
    void sound() { System.out.println("Animal Sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
Animal a = new Dog();
a.sound();   // Output: Bark
```

← Same method (sound) but different behavior.

10. What is compile-time vs runtime polymorphism?

Answer:

Compile-time Polymorphism	Runtime Polymorphism
• Achieved through method overloading. • Resolved at compile time. • Based on method signature (name + parameters). • Faster (early binding).	• Achieved through method overriding. • Resolved at runtime. • Based on object type. • Slower (late binding).

Example:

Compile-time (Overloading)

```
class Test {
    void show() { }
    void show(int x) { }
}
```

Runtime (Overriding)

```
class A { void show() { } }
class B extends A {
    @Override
    void show() { }
}
```

11. What is method overloading?

Answer:

- Method overloading means having multiple methods with the same name but different parameters (number, type, or order).
- It is compile-time polymorphism.

Example:

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
```

Note: Return type can be same or different. Overloading improves code readability.

12. What is method overriding?

Answer:

- Method overriding means redefining a method in the child class with the same name, same parameters and same return type.
- It is runtime polymorphism.

Example:

```
class Vehicle {
    void run() { System.out.println("Vehicle is running"); }
}
class Car extends Vehicle {
    @Override
    void run() { System.out.println("Car is running"); }
}
Vehicle v = new Car();
v.run();   // Output: Car is running
```

← Child class provides its own implementation.

Top 50 OOPS Interview Questions

PAGE 3 - QUESTIONS 13 TO 18

13. What is the difference between overloading and overriding?

Answer:

	Overloading	Overriding
Definition	Same class, multiple methods with same name but different parameters.	Subclass provides its own implementation of parent class method.
Where it happens	Within the same class	Between parent and child class
Method Signature	Must be different (number/type/order of params)	Must be same (name + params)
Return Type	Can be same or different	Must be same (or covariant in Java)
Access Modifier	Can be different	Must be same or more accessible
Binding	Compile-time (static binding)	Runtime (dynamic binding)

Pro Tip: Overloading is about "many forms", Overriding is about "new behavior".

14. Can we overload a method by changing only its return type?

Answer:

- No. In Java, method overloading is not possible by changing only return type.
- The compiler cannot differentiate methods having same name, same parameters and different return types.

Example:

```
class Test {
    int getValue() { return 10; }
    // Cannot overload
    double getValue() { return 20.5; } // Compile Error
}
```

Reason: Method signature (name + parameters) must be different for overloading.

15. Can static methods be overridden?

Answer:

- No. Static methods belong to the class, not to objects.
- They are resolved at compile time.
- We can hide a static method, but not override.
- Hiding happens when child class defines a static method with the same signature.

Example:

```
class Parent {
    static void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    static void show() { System.out.println("Child"); } // Hiding
}

Parent.show();   // Output: Parent
Child.show();    // Output: Child
```

← No overriding, this is method hiding.

16. Can private methods be overridden?

Answer:

- No. Private methods are not visible to subclasses.
- They cannot be overridden.
- If child class has a method with same signature, it is a new method, not overriding.

Example:

```
class Parent {
    private void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    private void display() { // New method, not overriding
        System.out.println("Child");
    }
}
```

← Both methods are independent.

17. Can constructors be overridden?

Answer:

- No. Constructors cannot be overridden.
- Constructors have the same name as class but they are not methods.
- They are used to initialize objects and are not inherited.

Example:

```
class Parent {
    Parent() { System.out.println("Parent Constructor"); }
}
class Child extends Parent {
    Child() { System.out.println("Child Constructor"); }
    // Not overriding, just another constructor
}
```

← Constructor chaining is done using super().

18. What is dynamic method dispatch?

Answer:

- Dynamic Method Dispatch is the process by which a call to an overridden method is resolved at runtime rather than compile time.
- It is a key feature of runtime polymorphism.
- The JVM determines which method to call based on the actual object.

Example:

```
class Animal {
    void sound() { System.out.println("Animal Sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}

Animal a = new Dog();
a.sound();  // Output: Bark (decided at runtime)
```

← Method call resolved at runtime based on actual object (Dog).

Top 50 OOPS Interview Questions

PAGE 4 - QUESTIONS 19 TO 24

19. What is an abstract class?

Answer:

- An abstract class is a class that cannot be instantiated and may contain abstract (unimplemented) methods and concrete (implemented) methods.
- It is declared using the "abstract" keyword.

Example:

```
abstract class Shape {
    abstract void draw();      // abstract method
    void area() {
        System.out.println("Calculating area");
    }
}
```

We cannot create object of Shape class.

20. What is an interface?

Answer:

- An interface is a blueprint of a class that contains abstract methods and constants only.
- It is declared using the "interface" keyword.
- A class implements an interface using "implements".

Example:

```
interface Drawable {
    void draw();      // abstract method
    int MAX = 100;  // public static final by default
}
```

← All fields in interface are public static final by default.

21. Abstract class vs interface - when would you use each?

Answer:

Basis	Abstract Class	Interface
Purpose	Provides base implementation and common behavior	Defines a contract (what to do)
Methods	Can have abstract + concrete methods	Only abstract methods (before Java 8)
Fields	Can have instance variables	Only public static final constants
Constructor	Can have constructors	Cannot have constructors
Multiple Inheritance	Not possible	Possible (a class can implement multiple interfaces)
When to use	When classes are closely related (IS-A)	When unrelated classes should follow same contract

Pro Tip: Use abstract class for sharing code.
Use interface for defining behavior / contract.

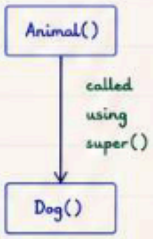
22. Can an abstract class have a constructor?

Answer:

- Yes, an abstract class can have constructors.
- The constructor is called when a subclass object is created.
- It is used to initialize the abstract class members.

Example:

```
abstract class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
    abstract void sound();
}
class Dog extends Animal {
    Dog() {
        super();  // calls Animal constructor
    }
    void sound() {
        System.out.println("Bark");
    }
}
```



Output:
Animal constructor

23. Can an interface have variables and methods?

Answer:

- Yes, interfaces can have variables and methods.
- Variables are implicitly public static final constants.
- Methods are public abstract by default (before Java 8).
- From Java 8+, interfaces can have default and static methods also.

Variables in Interface

```
interface Test {
    int COUNT = 10; // public static final
}
```

By default: public static final

Methods in Interface

```
interface Test {
    void show();          // public abstract
    default void display() {
        System.out.println("Default Method");
    }
    static void info() {
        System.out.println("Static Method");
    }
}
```

Java 8+ feature

24. Why does Java allow a class to implement multiple interfaces?

Answer:

- Java allows multiple inheritance of type (behavior) through interfaces, avoiding ambiguity.
- It promotes loose coupling and provides flexibility.
- A class can implement multiple interfaces using a comma separated list.

Example:

```
interface Flyable {
    void fly();
}
interface Swimmable {
    void swim();
}
class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck is flying"); }
    public void swim() { System.out.println("Duck is swimming"); }
}
```

← Duck can fly AND swim because it implements both interfaces.

Top 50 OOPS Interview Questions

PAGE 5 – QUESTIONS 25 TO 30

25. What is tight coupling vs loose coupling?

Answer:

- Tight coupling → Classes are highly dependent on each other. A change in one class affects others.
- Loose coupling → Classes are independent and interact through interfaces or abstractions.

Tight Coupling	Loose Coupling
• Strong dependency • Hard to maintain • Hard to test • Less flexible	• Weak dependency • Easy to maintain • Easy to test • More flexible & reusable

Pro Tip: Prefer loose coupling for scalable and maintainable code.

26. What is composition?

Answer:

- Composition is a strong HAS-A relationship.
- One class contains another class as its part.
- If the containing object is destroyed, the contained object also gets destroyed.

Example:

```
class Engine {
    void start() { System.out.println("Engine started"); }
}
class Car {
    private Engine engine;   // Engine is part of Car
    Car() { engine = new Engine(); }
    void start() { engine.start(); }
}
```



27. What is aggregation?

Answer:

- Aggregation is a weak HAS-A relationship.
- One class has another class as part, but both can exist independently.
- If the containing object is destroyed, the contained object can still exist.

Example:

```
class Department {
    String name;
}
class Employee {
    private Department dept; // Department is part of Employee
    Employee(Department d) { dept = d; }
}
```



28. Composition vs aggregation – what's the difference?

Answer:

Feature	Composition (Strong HAS-A)	Aggregation (Weak HAS-A)
Ownership	Strong ownership	Weak ownership
Dependence	Dependent object cannot exist without owner	Dependent object can exist without owner
Relationship	Life time is bound	Life time is independent
Symbol	Filled diamond (◆)	Hollow diamond (◇)
Example	Car – Engine	Employee – Department

Rule of Thumb: Use Composition when "part" cannot exist without "whole". Use Aggregation when it can exist independently.

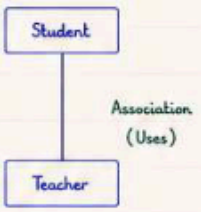
29. What is association?

Answer:

- Association is a general relationship between two classes.
- It represents the linkage or connection between objects.
- It can be one-to-one, one-to-many, many-to-many.

Example:

```
class Teacher {
    String name;
}
class Student {
    private Teacher teacher;   // Association
    Student(Teacher t) { teacher = t; }
}
```



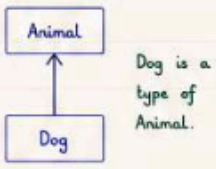
30. What is the difference between IS-A and HAS-A relationships?

Answer:

- IS-A → Inheritance relationship. One class "is a" type of another class.
- HAS-A → Composition / Aggregation relationship. One class "has a" another class.

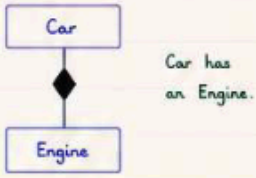
IS-A (Inheritance)

Example: Dog IS-A Animal



HAS-A (Composition / Aggregation)

Example: Car HAS-A Engine



Quick Summary

- Tight Coupling → Avoid
- Loose Coupling → Prefer
- Composition → Strong HAS-A (life bound)
- Aggregation → Weak HAS-A (life independent)
- Association → General link between classes
- IS-A → Inheritance | HAS-A → Composition/Aggregation

Top 50 OOPS Interview Questions

PAGE 6 - QUESTIONS 31 TO 36

31. What is the difference between this and super?

Answer:

- this → Refers to the current object.
- super → Refers to the immediate parent class object.

Example:

```
class Parent {
    int value = 10;
    void show() { System.out.println("Parent show"); }
}
class Child extends Parent {
    int value = 20;
    void display() {
        System.out.println(this.value);  // 20 (child)
        System.out.println(super.value); // 10 (parent)
        this.show();  // calls child class show() if exists
        super.show(); // calls parent class show()
    }
}
```

Key Point: Use this to access current object's members. Use super to access parent class members.

32. What is the difference between this() and super()?

Answer:

- this() → Calls another constructor of the same class.
- super() → Calls the constructor of the parent class.

Example:

this() Example

```
class Student {
  int id; String name;

  Student() {
    this(0, "Unknown");
  }

  Student(int id, String name) {
    this.id = id; this.name = name;
  }
}
```

super() Example

```
class Parent {
  Parent() {
    System.out.println("Parent");
  }
}
class Child extends Parent {
  Child() {
    super();  // calls Parent()
    System.out.println("Child");
  }
}
```

Note: In a constructor, this() or super() must be the first statement.

33. What is a constructor and why is it needed?

Answer:

- A constructor is a special method used to initialize objects.
- It has the same name as the class and no return type.
- It is called automatically when an object is created.
- It helps to initialize object's state.

Example:

```
class Book {
    String title;
    int price;

    // Constructor
    Book(String title, int price) {
        this.title = title;
        this.price = price;
    }
}
```

How it works:

1. Memory allocated
2. Constructor called
3. Object initialized
4. Reference returned

Without constructor, object may have default values.

34. What are default and parameterized constructors?

Answer:

- Default Constructor → No parameters.
- Parameterized Constructor → Takes one or more parameters.

Example:

Default Constructor

```
class Car {
    String model;
    int year;

  // Default Constructor
  Car() {
    model = "Unknown";
    year = 0;
  }
}
```

Parameterized Constructor

```
class Car {
    String model;
    int year;

  // Parameterized Constructor
  Car(String model, int year) {
    this.model = model;
    this.year = year;
  }
}
```

Pro Tip: If no constructor is written, Java provides a default no-arg constructor.

35. What is the difference between instance and static members?

Answer:

Feature	Instance Members	Static Members
Belong to	Object	Class
Access	Through object	Through class name
Memory	Each object has its own copy	One copy shared by all objects
Example	int age;	static int count;
Keyword	None	static
Use case	Object specific data	Common data for all objects

Example:

```
class Counter {
    int id;                 // Instance variable
    static int count = 0;   // Static variable

    Counter(int id) {
        this.id = id;
        count++;
    }
}
```

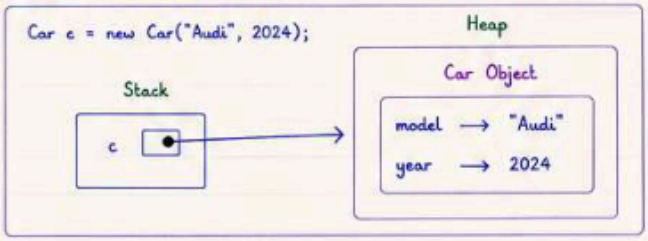
← count is shared by all objects.

36. What happens when an object is created in Java?

Answer:

1. Memory is allocated in heap for the object.
2. Instance variables are initialized with default values.
3. The constructor is called to initialize the object.
4. The reference of the object is returned.

Example:



Flow: new keyword → Memory allocation → Constructor call → Initialization → Reference returned

Top 50 OOPS Interview Questions

PAGE 7 - QUESTIONS 37 TO 42

37. What is the difference between == and equals()?

Answer:

- == checks reference equality (same memory location).
- equals() checks content equality (logical equality).

Example:

```
String s1 = new String("hello");
String s2 = new String("hello");
String s3 = "hello";
String s4 = "hello";

System.out.println(s1 == s2);     // false
System.out.println(s1.equals(s2)); // true
System.out.println(s3 == s4);     // true (same literal pool)
System.out.println(s3.equals(s4)); // true
```

Note: Always override equals() in custom classes to compare content.

38. Why should equals() and hashCode() be overridden together?

Answer:

- If two objects are equal (equals()), they must have the same hash code.
- Required for correct behavior in collections like HashMap, HashSet, HashTable.

Example:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Employee e = (Employee) o;
    return id == e.id;
}

@Override
public int hashCode() {
    return Objects.hash(id);
}
```

Key Point: If equals() is true, hashCode() must be same.

39. What is the toString() method?

Answer:

- toString() returns a string representation of the object.
- By default, it returns ClassName@hashcode.
- We override it to provide meaningful information.

Example:

```
class Employee {
  int id; String name;
  Employee(int id, String name){ this.id = id; this.name = name;}

  @Override
  public String toString() {
     return "Employee{" + "id=" + id + ", name='" +
          name + "'}";
  }
}
System.out.println(new Employee(1, "Aman"));
  // Output: Employee{id=1, name='Aman'}
```

Pro Tip: Helpful for debugging and logging.

40. What is an immutable class?

Answer:

- An immutable class is a class whose objects cannot be modified once created.
- All fields are final, no setters, and defensive copying if needed.

Example:

```
class ImmutablePerson {
   private final String name;
   private final int age;

   public ImmutablePerson(String name, int age) {
      this.name = name;
      this.age = age;
   }

   public String getName() { return name; }
   public int getAge() { return age; }
}
```

Benefits:

- Thread-safe
- Easy to share
- No side effects
- Secure

41. How would you create an immutable class in Java?

Answer:

Steps:

1. Declare class as final (so it can't be inherited).
2. Make all fields private and final.
3. Do not provide setter methods.
4. Initialize all fields through constructor.
5. If mutable objects exist, return defensive copies.

Example:

```
public final class Address {
  private final String city;
  private final String state;

  public Address(String city, String state) {
     this.city = city;
     this.state = state;
  }

  public String getCity() { return city; }
  public String getState() { return state; }
}
```

Extra Tip: Avoid exposing mutable fields like List, Date, etc. Return copies.

42. Why is String immutable in Java?

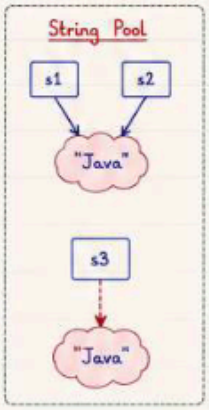
Answer:

- String is immutable to provide security, caching and performance.
- Immutable Strings are stored in String Pool.
- Important for multithreading and hashing.

Example:

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");

System.out.println(s1 == s2); // true (same pool)
System.out.println(s1 == s3); // false (new object)
```



Why immutable?

→ Security (String used in network, file paths, class loading)
→ String Pool for memory optimization
→ Thread-safe

Top 50 OOPS Interview Questions

PAGE 8 - QUESTIONS 43 TO 47

43. What is upcasting and downcasting?

Answer:

- Upcasting → Converting child reference to parent reference (Implicit).
- Downcasting → Converting parent reference to child reference (Explicit).

Example:

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    void bark() { System.out.println("Bark"); }
}
// Upcasting
Animal a = new Dog();    // Implicit
a.sound();    // Output: Animal sound

// Downcasting
Dog d = (Dog) a;    // Explicit
d.bark();    // Output: Bark
```

Key Point:
Downcasting must be done carefully using instanceof to avoid ClassCastException.

```
Example:
if (a instanceof Dog) {
    Dog d = (Dog) a;
    d.bark();
}
```

44. What is instanceof and when should you use it?

Answer:

- instanceof is an operator used to test whether an object is an instance of a class, subclass or interface.
- Use it before downcasting to avoid ClassCastException.

Example:

```
Animal a = new Dog();
if (a instanceof Dog) {
    Dog d = (Dog) a;
    d.bark();
} else {
    System.out.println("Not a Dog object");
}
```



Pro Tip: Always use instanceof before downcasting. Never assume type blindly.

45. Can we achieve abstraction without using an abstract class?

Answer:

- Yes. We can achieve abstraction using interfaces.
- Interfaces allow us to define behavior (methods) without implementation.

Example:

```
interface Shape {
    void draw();    // Abstract method
}
class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}
// No abstract class used, still abstraction achieved
```

Key Point:
Use interface when you need 100% abstraction without sharing state.

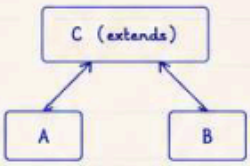
46. Can an interface extend another interface?

Answer:

- Yes. An interface can extend one or more interfaces.
- This helps in achieving multiple inheritance of type.

Example:

```
interface A {
    void methodA();
}
interface B {
    void methodB();
}
interface C extends A, B {
    void methodC();
}
class Demo implements C {
    public void methodA() { }
    public void methodB() { }
    public void methodC() { }
}
```



Benefit:
Helps in building hierarchy of interfaces and reusing behavior.

47. Can an abstract class implement an interface without implementing all its methods?

Answer:

- Yes. An abstract class can implement an interface.
- It can choose to implement some methods and leave others unimplemented (abstract).

Example:

```
interface Payment {
    void pay();
    void refund();
}
abstract class BasePayment implements Payment {
    public void pay() {
        System.out.println("Payment done");
    }
    // refund() not implemented
}

class CreditCardPayment extends BasePayment {
    public void refund() {
        System.out.println("Refund to Credit Card");
    }
}
```

Key Point:
Abstract class can be partial implementation of interface.
Concrete class must implement remaining methods.

Summary of Page 8

Question	Key Concept	Quick Note
43	Upcasting & Downcasting	Upcasting is implicit, Downcasting is explicit & use instanceof
44	instanceof	Used to check object type before downcasting
45	Abstraction without Abstract Class	Interfacen provide 100% abstraction.
46	Interface extends Inherface	Yes, supports multiple inheritance of type
47	Abstract class implements Interface	Can implement partially, subclass completes

Top 50 OOPS Interview Questions

PAGE 9 - QUESTIONS 48 TO 50

48. What is upcasting and downcasting?

Answer:

- Upcasting → Converting child reference to parent reference (Implicit).
- Downcasting → Converting parent reference to child reference (Explicit).

Example:

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    void bark() { System.out.println("Bark"); }
}
public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        // Upcasting
        Animal a = d;    // implicit
        a.sound();       // OK

        // Downcasting
        Animal a2 = new Animal();
        // Dog d2 = (Dog) a2;  // Compile-time OK,
        // but throws ClassCastException at runtime
    }
}
```

Key Point:

Upcasting is safe and implicit.

Downcasting must be done carefully using instanceof to avoid runtime exception.

Example:

```
if (a instanceof Dog) {
    Dog d3 = (Dog) a;
    d3.bark();
}
```

49. What is instanceof and when should you use it?

Answer:

- instanceof is an operator used to test whether an object is an instance of a class, subclass or interface.
- Use it before downcasting to make code safe.

Example:

```
class Animal { }

class Dog extends Animal {
    void bark() { System.out.println("Bark"); }
}
class Test {
    public static void main(String[] args) {
        Animal a = new Dog();
        if (a instanceof Dog) {
            Dog d = (Dog) a;    // safe downcast
            d.bark();
        } else {
            System.out.println("Not a Dog");
        }
    }
}
```

Why use instanceof?

- ✓ Prevents ClassCastException.
- ✓ Helps in safe downcasting.
- ✓ Improves code robustness.

50. Can we achieve abstraction without using an abstract class?

Answer:

- Yes. We can achieve abstraction using interfaces.
- Interfaces allow us to define behavior (methods) without implementation.

Example:

```
interface Shape {
    void draw();  // Abstract method
}
class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle implements Shape {
    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}
```

Key Point:

Using interfaces, we achieve abstraction without using abstract class.

Benefits:

- Multiple inheritance of type (interfaces).
- Only behavior (contract) is exposed.
- Flexible & loosely coupled.

Summary of Page 9

Question	Key Concept	Quick Note
48	Upcasting & Downcasting	Upcasting is implicit & safe. Downcasting is explicit & must be done carefully.
49	instanceof	Used to check object type before downcasting. Prevents ClassCastException.
50	Abstraction without Abstract Class	Yes, using interfaces. Define behavior without implementation. Provides flexibility.

Quick Recap

- Upcasting → Child to Parent (implicit)
- Downcasting → Parent to Child (explicit)
- instanceof → Safe type check before downcasting
- Abstraction can be achieved using interfaces
- Interfaces define contract, not implementation
- Leads to loose coupling & flexible design